The Agent Operator's Handbook

A Field Guide to Dispatching, Steering, and Recovering Multiple AI Coding Agents at Once

Causeway Services

A 561 Group book

The Agent Operator's Handbook
A Field Guide to Dispatching, Steering, and Recovering Multiple AI Coding Agents at Once

Copyright © 2026 Causeway Services. All rights reserved in whatever of this book copyright protects.

This book was written by AI agents working for the 561 Group and proofread by a separate AI agent. The PDF is set in Roboto Slab and Noto Sans.

A 561 Group book.
shop.561.group

First published September 2026.
Edition `ni:///sha-256;MFd6qxlsUQL7IlF1lGdw3Zd-CfXIcIlQc9Px4ru0hXo`

Contents

About this book **7**

Who this book is for . . . 7

How this book is organized . . . 7

A note on honesty . . . 8

1 Chapter 1: The Complete Brief **9**

1.1 What a subagent actually knows . . . 9

1.2 The four things every brief needs . . . 9

1.3 Why “verbatim evidence” matters more than a summary . . . 10

1.4 Deciding whether to dispatch at all . . . 11

1.5 A worked example . . . 11

1.6 Briefs for ambiguous, non-technical decisions . . . 12

1.7 The habit to build . . . 13

2 Chapter 2: The Fan-Out Anti-Pattern **15**

2.1 More agents is not more thoroughness . . . 15

2.2 The question that decides it . . . 15

2.3 The three-part test . . . 15

2.4 What a legitimate split looks like . . . 16

2.5 “One agent per directory” is not a plan . . . 17

2.6 A worked example . . . 17

2.7 Stating the decision a split would change . . . 18

3 Chapter 3: Steering a Running Agent **19**

3.1 A message queues; it does not interrupt . . . 19

3.2 Never say “done” before it’s true on disk . . . 19

3.3 The one-nudge rule for a stuck agent . . . 20

3.4 Stopping an agent, and its children . . . 21

3.5 A worked example . . . 21

3.6 What comes next . . . 22

4 Chapter 4: Recovering an Interrupted Commission **23**

4.1 Absence is not evidence of failure . . . 23
4.2 The three-step recovery, in order . . . 23
4.3 Why "redo the whole thing" feels safer than it is . . . 24
4.4 A worked example . . . 24
4.5 When the agent itself, not just your connection to it, is the uncertain part . . . 25
4.6 The forward test . . . 26

5 Chapter 5: Receiving a Report 27
5.1 The report is all that comes back . . . 27
5.2 Send back anything missing verbatim evidence . . . 27
5.3 Absence of counter-evidence is not proof of absence . . . 27
5.4 "I delegated this" is not a result . . . 28
5.5 Verify the one load-bearing claim yourself . . . 28
5.6 Calibrate ceremony to actual consumers . . . 28
5.7 "Blocked" needs to say what a competent outsider would try . . . 29
5.8 Reconciling multiple reports before acting on any one . . . 29
5.9 A worked example . . . 29
5.10 A second worked example: the confidently wrong "no issues found" . . . 30
5.11 What this buys you . . . 30

6 Chapter 6: Receipts as Inter-Session Memory 33
6.1 The problem receipts solve . . . 33
6.2 What a receipt is, and what it isn't . . . 33
6.3 The minimal shape of a receipt . . . 34
6.4 A worked example . . . 34
6.5 When a reconciled finding set is itself a receipt . . . 35
6.6 Falsehood is removed, never annotated . . . 35
6.7 Receipts for decisions, not just findings . . . 36
6.8 Building the habit . . . 37

7 Chapter 7: Case Study — From Collision to Coordination 39
7.1 A note on this chapter . . . 39
7.2 The setup . . . 39
7.3 The collision . . . 40
7.4 Why this matches Chapter 2 and Chapter 5, not just bad luck . . . 40
7.5 The resolution: division by message . . . 41
7.6 The rule this incident supports . . . 41
7.7 Applying this to your own concurrent sessions . . . 42

8 Chapter 8: Building Your Own Operating Discipline 43
8.1 Why a checklist, and why now . . . 43
8.2 Before you dispatch . . . 43
8.3 While an agent is running . . . 44
8.4 When something interrupts you . . . 44
8.5 When a report lands . . . 44
8.6 At the end of a session . . . 45
8.7 Adopting this gradually . . . 45
8.8 The test that tells you it's working . . . 46

9 Chapter 9: Field Diagnoses — Common Situations, Quickly Placed 47
9.1 "The agent said it fixed the bug, but the bug is still there." . . . 47
9.2 "Two agents wrote to the same file and now it's a mess." . . . 47
9.3 "I came back to my laptop and I don't know if the agent finished." . . . 48
9.4 "The agent kept saying 'still waiting' for twenty minutes." . . . 48
9.5 "I stopped the agent, but something is still editing my files." . . . 48
9.6 "A report says a task is blocked, and I don't know if that's really true." . . . 49
9.7 "I have six agent reports and I don't know what to believe." . . . 49
9.8 "I can't remember what I already figured out about this bug three days ago." . 49
9.9 "A document in my notes contradicts itself — one section says X, another says the opposite." . . . 50
9.10 "I'm about to split a big task across five agents — should I?" . . . 50
9.11 "The agent asked to confirm something I thought I'd already told it." . . . 50
9.12 "A teammate's agent reverted something my agent had just finished." . . . 51
9.13 Using this chapter . . . 51

10 Chapter 10: Scaling This Discipline to a Team 53
10.1 The brief becomes a shared artifact, not a private habit . . . 53
10.2 Path manifests stop being optional . . . 54
10.3 Receipts become the team's shared memory, not just yours . . . 54
10.4 Reconciling reports across people, not just across agents . . . 54
10.5 What doesn't change . . . 55
10.6 A team using more than one kind of agent . . . 55
10.7 The single habit worth building first, on a team . . . 56

11 Glossary 57

12 Appendix: Five Templates 59
12.1 1. The brief template . . . 59

12.2 2. The fan-out decision checklist 60
12.3 3. The recovery runbook 60
12.4 4. The minimal receipt format 61
12.5 5. The coordination protocol (path manifests) 61

About this book

Who this book is for

You already pay for an agentic coding subscription — Claude Code, Codex, or something like them — and you've started running more than one agent at a time, because one agent working alone leaves obvious throughput on the table. This book assumes exactly that starting point and nothing more. It does not teach you what an AI coding agent is, how to write a prompt, or how to pick a model. It teaches the one thing that almost nobody teaches explicitly: the operating discipline that keeps two or more agents from costing you more than they save.

If you have ever had an agent quietly overwrite a colleague's uncommitted work, watched a "done" report turn out to be wrong once you actually opened the file, or spent an afternoon untangling what three parallel agents did to the same directory, this book is about you.

How this book is organized

Chapters 1 through 6 each teach one discipline: writing a complete brief, recognizing the fan-out anti-pattern, steering a running agent, recovering an interrupted commission, receiving a report without accepting an unverified claim, and using dated receipts as memory between sessions. Chapter 7 is a single worked case study — a real, dated incident where two concurrent agent sessions collided on one shared, uncommitted repository — walked from the collision to its resolution. Chapter 8 pulls the six disciplines into one operating checklist you can actually run day to day, and Chapter 9 collects twelve common situations into quick diagnoses you can check a live problem against. Chapter 10 extends the whole discipline from one operator to a small team. A glossary and an appendix of five reusable templates — a brief template, a fan-out decision checklist, a recovery runbook, a minimal receipt format, and a coordination protocol for two or more people (or agents) sharing one repository — close out the book.

Chapters 1 through 6 each include at least one worked example grounded in an ordinary software project — nothing here requires you to understand any particular company's in-

ternal tooling or vocabulary. The case study in Chapter 7 is real and dated; every other example is a composite built to be representative rather than a transcript of one specific session.

A note on honesty

None of the discipline in this book is theoretical advice arrived at by thinking hard about what agents *should* do. Every rule traces back to a specific failure: a redone task, a silently overwritten file, a "done" that wasn't true on disk. The rule is what would have prevented the failure, written down after the failure happened, not before. Where a worked example is illustrative rather than a literal transcript, this book says so. Where it is a real incident, this book says that too.

Chapter 1

Chapter 1: The Complete Brief

1.1 What a subagent actually knows

Start with an uncomfortable fact about how these tools work: when you dispatch a subagent, it starts with nothing. It has not read your earlier messages in this conversation. It does not know what you tried five minutes ago, what you ruled out, or what "obviously" you meant when you typed four words and hit enter. It knows exactly, and only, what is in the text you hand it, plus whatever it can discover by reading files, running commands, and calling tools once it starts working.

This sounds obvious once it's stated. In practice, almost everyone violates it constantly, because writing to a subagent *feels* like writing to a colleague. You've been in the conversation. The colleague metaphor leaks context you don't realize you're leaning on: shared history, shared assumptions about what "clean up the auth module" means, a shared sense of what's out of bounds. A subagent has none of that. It is, for the purposes of the task you're about to hand it, a highly capable stranger who read only the paragraph you wrote.

The single biggest lever you have over whether a dispatched agent produces the thing you wanted is whether the brief you wrote is complete. A complete brief does not guarantee good work — an agent can still misjudge something, misread a file, or make a reasonable call you'd have made differently. But an *incomplete* brief guarantees confidently wrong work, because the agent will not stop and ask; it will fill the gap with its own best guess and proceed as if that guess were a given fact. The guess is usually plausible. It is also usually not what you meant.

1.2 The four things every brief needs

A complete brief states four things, in whatever order reads naturally, but all four have to be there:

1. The requested outcome. Not the action — the outcome. "Look at the payment webhook

handler" is an action with no outcome attached; the agent doesn't know when it's done looking, or what it's looking *for*. "Find out whether the payment webhook handler correctly verifies the provider's signature before processing a refund event, and report the exact line where verification happens or is missing" is an outcome. The agent knows when it has an answer.

2. The exact scope. Name the files, directories, or boundary explicitly. "The auth module" is not a scope; `src/auth/` is. If the task touches configuration, tests, or a sibling package, say so explicitly — an agent that isn't told a boundary will guess one, and it may guess wider or narrower than you intended. When you're running more than one agent at once (which is most of what this book is about), an unstated scope is how two agents end up editing the same file without either of them knowing.

3. The permitted operation. This is the one people skip most often, and it's the one that causes the most expensive mistakes. State plainly whether the agent may only *investigate* (read-only — report back, touch nothing), propose a *patch for review* (write the change but don't finalize it — a diff you'll look at before it lands), or make an *authorized edit* (go ahead and change the files). These are three different levels of trust, and an agent that isn't told which one it has will sometimes pick the most helpful-seeming one on its own — which usually means editing, because editing looks more like finishing the job than reporting back does. If you wanted an investigation and get an edit instead, that's not the agent misbehaving; that's a brief that never said "read-only."

4. What "done" measures. A fixed, checkable standard — not a vague sense of quality. "Make sure the tests pass" is vague until you name *which* tests, with what command, and what count you expect. "Run `npm test` in `packages/billing/` and report the exact pass/fail count" is checkable. A brief that ends with "make sure it's good" is a brief that lets an agent declare victory on its own say-so, and that is exactly how a false "done" gets reported back to you.

1.3 Why "verbatim evidence" matters more than a summary

Ask for exact commands and exact output, every time, in every report you require back — a test count, a line count, an exit code, the literal output of the command that proves the claim. Not "the tests pass now," but `node --test` producing `# pass 47` and `# fail 0`.

The reason this matters isn't paranoia about agents lying. It's that "the tests pass now" is a summary, and summaries round up. An agent that ran the test suite, saw 44 pass and 3 fail, fixed one of the three, and ran out of budget on the other two will very naturally write

"fixed the failing tests" if you didn't ask for the count — not because it's being dishonest, but because "fixed the failing tests" is a true statement about *effort*, and it reads, to a skimming eye, like a true statement about *outcome*. The exact count — 45 pass, 2 fail — tells you the actual state in a way no adjective can. Demand the number every time, and you will catch this long before it costs you a broken deploy.

1.4 Deciding whether to dispatch at all

The most complete brief in the world is still wasted if the dispatch itself wasn't warranted. Before you write a brief, ask one question: can the result actually change your next decision? If you already know the answer — you're dispatching to double-check something you're confident about, or to "be thorough" on a task you could finish yourself in the same amount of time it takes to write the brief — you're paying tokens and wall-clock time for a report you were going to act on the same way regardless of what it said. That's not caution. That's overhead dressed as caution.

This becomes especially important once you're running several agents at once, which is the whole subject of Chapter 2 — the temptation to spin up "one more agent, just in case" scales with how easy dispatching has become, and the discipline that keeps it from ballooning is asking this question every single time, not just the first time.

1.5 A worked example

Suppose you're a solo developer building a subscription billing API. You want to know whether a recent refactor of the invoice-generation code broke anything, and you also want a second agent to add a new endpoint for partial refunds. Here is an incomplete brief for the first task, followed by a complete one.

Incomplete:

> Check if the invoice refactor broke anything.

This tells the agent nothing about which files changed, what "broke" means (a compile error? a test failure? a behavior change a human would notice but no test catches?), whether it may fix what it finds, or how it should report back. A capable agent will do *something* useful with this — probably run the test suite and report whatever it sees — but you've delegated every judgment call about what "checking" means to the agent's guess, and you won't know which guesses it made unless you read its full transcript.

Complete:

> Outcome: determine whether the invoice-generation refactor in commit a3f9e21 changed the computed total for any existing invoice fixture. Scope: `src/billing/invoices/` and its test fixtures in `test/fixtures/invoices/`; do not touch `src/billing/refunds/` — a second agent is working there concurrently. Permitted operation: read-only investigation. Do not edit any file. Done: run `npm test -- invoices` and report the exact pass/fail count. If any test fails, report the exact fixture name, the expected total, and the actual total the refactored code now produces. If all tests pass, additionally diff the refactor's arithmetic against the pre-refactor version at the parent commit and report whether the rounding behavior for fractional cents changed, since that's the one case the existing fixtures don't cover.

Notice what the complete version does that the incomplete one doesn't: it names a specific commit, draws an explicit scope boundary that protects a concurrently-working second agent's territory, states plainly that this is read-only, gives an exact command and requires the exact count, and — because the person writing the brief knows something the agent doesn't (that the existing fixtures have a coverage gap around rounding) — states that knowledge explicitly instead of hoping the agent discovers it independently. A subagent cannot use context you have but never wrote down.

1.6 Briefs for ambiguous, non-technical decisions

Not every brief is about code. Once you're dispatching agents regularly, you'll find yourself using them for judgment-adjacent tasks too — draft a customer response, summarize a batch of support tickets, propose names for a new feature flag. These feel less "brief-shaped" than a scoped code change, and that feeling is exactly why they're worth extra care: an ambiguous task invites an ambiguous brief, and an ambiguous brief is where the agent's guess is most likely to diverge from what you actually wanted, because there's no test suite or exit code to catch the divergence for you.

The same four elements still apply, they just take different shapes. The outcome is still a concrete deliverable ("a two-sentence response a support lead could send as-is," not "help with this ticket"). The scope is still a boundary ("only the eleven tickets tagged `billing-dispute` from the past week," not "the support queue"). The permitted operation still matters — draft for review is almost always right here, since a customer-facing message going out without a human reading it first is a much higher-stakes version of the "helpfully edited when it should have only investigated" mistake from earlier in this chapter. And "done" still needs a standard, even for something as soft as tone: "matches

the company's existing reply templates in `docs/support-templates/`, addresses the specific complaint, and does not promise a refund amount" is checkable in a way "sounds good" is not.

The instinct to relax the brief's rigor for a "softer" task is understandable and wrong. Soft tasks are exactly where an agent's plausible-sounding guess is hardest for you to catch at a glance, because there's no failing test to flag it — only your own read of whether the output feels right, which is a much weaker check than a command's exit code.

1.7 The habit to build

Before you dispatch, read your own brief back and ask: if I gave this exact text to a competent stranger who has never seen this codebase and will never talk to me again before reporting back, would they know *only* what I intended them to know, and nothing they'd have to guess? If there's a gap, that gap is a decision you're silently delegating to whatever the agent guesses. Sometimes that's fine — not every ambiguity matters. But it should be a decision you made on purpose, not one that happened because you were in a hurry.

The next chapter covers a mistake that happens on the other side of this same instinct: not an incomplete brief, but *too many* briefs, written because splitting a task across several agents felt like being thorough when it wasn't actually changing the answer.

Chapter 2

Chapter 2: The Fan-Out Anti-Pattern

2.1 More agents is not more thoroughness

Once you've dispatched a handful of subagents and watched them come back with real, useful results, a specific temptation shows up: a broad task arrives, and running several agents against it in parallel *feels* like the thorough version of doing it — surely eight agents each reading a slice of the codebase will catch more than one agent reading the whole thing sequentially?

Usually not. This is the fan-out anti-pattern, and it is worth a whole chapter because it is expensive in a way that's easy to miss: every additional dispatch costs real tokens and real wall-clock time, and if the split doesn't actually change what you conclude, you've paid that cost for nothing. Worse, a careless fan-out multiplies the failure modes from Chapter 1 — now you have several incomplete briefs instead of one, several agents each guessing at scope boundaries, and a reconciliation problem waiting for you once the reports land.

2.2 The question that decides it

Before splitting a task across multiple agents, ask exactly one question: **does splitting this into N agents change the answer, or would one agent working through the material sequentially reach the same conclusion for a fraction of the cost?**

If the honest answer is "one agent would reach the same conclusion," the fan-out is not adding thoroughness. It's adding overhead dressed as thoroughness — the same failure mode as an unwarranted single dispatch from Chapter 1, just multiplied.

2.3 The three-part test

Fan out only when all three of these hold:

1. **The resulting lots (the pieces you divide the task into) are few enough to name indi-**

vidually. If you can't write, in the brief itself, a short list of exactly what each agent owns — "Agent A: the payments service; Agent B: the notifications service; Agent C: the admin dashboard" — you don't have a real split, you have a vague intention to divide work that you're hoping the agents will sort out among themselves. They won't sort it out; they'll each do their own version of the whole task, or collide on the parts that overlap.

2. Each lot is genuinely disjoint. No two agents should read or touch the same file. This is not a nicety — it's the difference between parallel work and a race condition. Two agents editing the same file concurrently is exactly how confirmed, correct work gets silently clobbered by a second agent that didn't know the first one had already fixed it.

3. Each lot needs independently gathered evidence. Not a different slice of the *same* search — a genuinely separate body of evidence that one agent reading sequentially would still have to gather separately anyway. If three agents would all end up running the same three commands and reading the same five files, just from different starting points, that's one task wearing a three-agent costume.

2.4 What a legitimate split looks like

A split by genuinely separate capability passes the test. If your product runs on three physically separate deployment targets — say, an iOS app, an Android app, and a web backend — and you need to know whether a recent API contract change broke each client, that's three disjoint codebases, three separate build toolchains, and three bodies of evidence that don't overlap. One agent per client is a real split: each lot is nameable, disjoint, and needs its own evidence (you can't learn whether the iOS app broke by reading the Android app's code).

Contrast this with "clean up all the outdated comments in the codebase." Splitting that into "one agent per directory" looks like the same shape — nameable lots, disjoint file territory — but fails the third leg of the test. Every one of those agents is doing the identical task (scanning for outdated comments) against the identical kind of evidence (reading files and judging staleness), just on a different slice of the same corpus. One agent walking the whole tree sequentially reaches the same conclusions; splitting it doesn't change what you learn, it just turns one bounded read into N bounded reads that you now have to reconcile.

2.5 "One agent per directory" is not a plan

This is worth stating bluntly because it's the single most common disguised fan-out: taking a broad, singular task and mechanically dividing it by directory, by file type, or by "give everyone something to do," rather than by a genuine difference in what evidence each lot requires. It looks organized. It produces a tidy-looking brief with several named agents and clear boundaries. But if the underlying task was one coherent judgment — "does this codebase still describe itself accurately," "is this pattern used consistently" — dividing the *reading* of it across agents does not divide the *judgment*. You still have to reconcile N partial views into one coherent answer, and that reconciliation cost is real and often larger than the cost you were trying to save by parallelizing.

There's a real, documented version of this failure that's worth internalizing as the canonical warning shape: a broad prose cleanup task spawned eight parallel agents on a single session's own initiative, burning meaningful tokens and time on cosmetic cleanup, while the actual product the team was building sat with zero new code written that day. Nobody decided "we should spend this budget on parallel prose cleanup instead of building the product" — it happened because splitting felt like the responsible, thorough move in the moment, and nobody asked the one question first.

2.6 A worked example

Suppose you maintain a monorepo with fourteen internal packages and you want to know which ones still import a deprecated logging library ahead of removing it. The fan-out instinct says: fourteen packages, fourteen agents, one each.

Apply the test. Is this genuinely disjoint, nameable, independent evidence? No — "does this package import the deprecated logging library" is a single grep-shaped question with one search pattern and one answer format, repeated fourteen times. One agent (or, more precisely, one command) answers it for all fourteen packages in less time than it takes to write fourteen briefs:

```
grep -rl "require(['\"]old-logger['\"])" packages/*/src
```

Now suppose the real task is different: you want each of three teams that own three genuinely separate services (billing, notifications, search) to migrate their own service off the deprecated library, because the migration in each service requires different judgment calls about what replaces the old logger's specific call sites, and those judgment calls need each service's own context. That's a real split — three disjoint codebases, three bodies

of service-specific judgment, three agents that don't need to read each other's territory. Name them individually in the brief, give each its exact scope, and dispatch three, not fourteen.

2.7 Stating the decision a split would change

Before you write a multi-agent brief, write one sentence: "splitting this into N agents changes the answer because ___." If you can't finish that sentence honestly, don't split it — do it as one agent, or do it yourself if it's small enough. This single habit is the whole chapter, compressed: the fan-out anti-pattern survives on the unstated assumption that more parallelism is free. It isn't. Price it before you spend it.

Chapter 3 turns to what happens once an agent — one agent, correctly dispatched — is actually running, and something needs to change about what it's doing before it finishes.

Chapter 3

Chapter 3: Steering a Running Agent

3.1 A message queues; it does not interrupt

Once an agent is running, you can send it a message to change what it does next — a correction, new information, a redirection. It's worth understanding precisely what that message does mechanically, because the wrong mental model here causes real mistakes.

In most current tools, a steering message queues for the agent's *next tool call*. It does not interrupt whatever the agent is doing mid-action. If the agent is in the middle of running a long build, your message waits until that build finishes and the agent goes to decide its next step; it does not stop the build. If the agent is halfway through writing a file, your message doesn't yank the pen out of its hand.

This matters most when the thing you need to tell the agent is urgent — the environment underneath it just changed in a way that invalidates what it's doing. If a file it's about to edit just got moved by another process, if a shared resource it's about to write to just got claimed by someone else, if an assumption it's operating on just became false — send that message immediately, and understand that it will still take effect only at the agent's next natural decision point, not instantly. Don't mistake "I sent the correction" for "the agent has already incorporated it." Watch for the agent's next action to confirm it landed.

3.2 Never say "done" before it's true on disk

This rule runs in both directions, and it deserves to be stated as starkly as possible: never tell an agent — and never tell a person, in a status update, a message to a teammate, or your own notes — that something is done before it is actually done on disk. Not "should be done by now." Not "probably finished." Done means you checked, and the file, the test result, the deployed state actually reflects the claim.

The reason this rule earns its own paragraph rather than being folded into general professionalism: a premature "done" doesn't just create minor embarrassment when it turns

out to be false. It propagates. If you tell a second agent "the migration script is already updated, build on top of it," and it isn't actually updated yet, the second agent will now build its own work on a false premise, and by the time anyone notices, you have two pieces of work that need to be untangled instead of one clean sequence. Whatever reads your claim next — a person, another agent, your own future self — inherits the error at one remove, where it's much harder to see.

3.3 The one-nudge rule for a stuck agent

Sometimes an agent appears to be stuck — idling, repeating a phrase like "waiting for the build to finish," not making visible progress. The instinct is to nudge it: send another message, ask if it's still there, try rephrasing the request. Resist repeating this. An agent that's genuinely stuck won't be un-stuck by a second or third nudge that carries no new information; you're just spending more tokens hoping the same input produces a different output.

Instead, apply the one-nudge rule:

1. **Check whether the underlying process is actually still alive.** If the agent is waiting on a build, a test run, or a long command, look at whether that process is still running, hung, or already finished with output the agent hasn't noticed yet.
2. **Kill it if it's hung.** A process that's genuinely wedged isn't going to un-wedge itself; killing it and letting the agent retry is faster than waiting indefinitely on the chance it recovers.
3. **Try the step once, in the foreground, yourself if needed.** This tells you definitively whether the step itself is broken (in which case the agent was right to be stuck) or whether it was an artifact of how the agent was waiting (in which case a clean retry unblocks it).
4. **Report the real outcome and stop.** Whichever of the above turns out to be true, act on that finding — redirect the agent, fix the underlying problem, or accept that this step needs a different approach — rather than sending a fifth identical nudge.

This procedure costs you a small amount of investigation up front, and it saves you from the much larger cost of an agent (or a whole afternoon) stuck in a loop that no amount of "are you there?" was ever going to break.

3.4 Stopping an agent, and its children

If you need to stop a running agent, stop it explicitly, using whatever your tooling provides for terminating a task — don't just close the conversation and assume the underlying process dies with it. And be aware of a subtlety that catches people off guard: if the agent you stopped had itself spawned child agents (a common pattern when one agent delegates sub-pieces of its own task), those children may keep running as independent tasks even after their parent is gone. Stopping the parent does not automatically stop its children. If you don't want them continuing — and usually you don't, especially if the reason you stopped the parent was that its whole plan turned out to be wrong — check for and stop the children individually.

This is not a hypothetical edge case. It's exactly the kind of thing that produces the "wait, why is there still an agent editing this file, I thought I stopped everything" moment, usually discovered only because something changed on disk that nobody expected.

3.5 A worked example

You're running two agents concurrently on a subscription billing codebase: one implementing a new discount-code feature, one investigating a reported bug in the invoice PDF renderer. Twenty minutes in, you realize the discount-code feature needs to change the shape of the `Invoice` record — a field the PDF-renderer investigation agent is currently reading to understand the bug.

The correct move: send the discount-code agent a steering message immediately, telling it the field shape change means it now also needs to coordinate with (or wait for) the renderer investigation, since that agent's read-only findings might reference the field it's about to change shape. You cannot assume this message takes effect mid-edit if the agent happens to already be writing the schema change when your message arrives — it will pick it up at its next tool call, which for an agent mid-edit might already be the completion of that specific write. Watch its next action. If it's already past the point where the warning would have mattered, you may need to stop it, assess what it actually wrote, and decide from measured state rather than from what you intended the message to prevent.

The wrong move, and a common one: assume the message landed instantly, tell the renderer-bug agent "the field shape is already updated, use the new shape," and move on — before confirming the discount-code agent actually finished that change. If it hadn't, the renderer agent is now working from a false "done" claim, and you've created exactly the propagating error described earlier in this chapter.

3.6 What comes next

Steering handles an agent that's still running. The next chapter handles the harder case: an agent whose report never arrived at all, because you lost the thread, the session restarted, or something else interrupted the connection between you and it. The instinct there is almost always wrong, and getting it right is worth a chapter of its own.

Chapter 4

Chapter 4: Recovering an Interrupted Commission

4.1 Absence is not evidence of failure

Sooner or later, you'll dispatch an agent and its report will never arrive — your session crashes, your laptop sleeps, the connection drops, you close the wrong window, or you simply lose track of which of six open threads was waiting on what. When you come back, there's a gap where a report should be, and a real, working agent that may or may not still be doing something to your files.

The instinct in that moment is almost always wrong, and it's worth naming precisely why. The gap *feels* like evidence that something failed — surely if the work had gone fine, you'd have gotten the report? But the report's absence tells you nothing about the work. It tells you only that the report didn't reach you, which can happen for reasons that have nothing to do with whether the underlying task succeeded. The agent may have finished cleanly ten minutes after you lost the thread. It may still be running right now. Treating its absence as proof of failure and acting on that assumption is how a solo operator ends up with two agents writing to the same files, or a completed, correct piece of work getting thrown away because nobody checked before starting over.

4.2 The three-step recovery, in order

1. Find the agent before you touch anything. List the original dispatched task and any children it may have spawned, and obtain one current status or terminal report from that exact agent — not a new one, not a guess, the actual one you lost track of. If it's still running, your job right now is to observe it (and steer it, per Chapter 3, if something needs correcting) — not to start a second, competing agent on the same files "just to be safe." If it already finished, read its actual report before you do anything else; don't assume you know what it did because you know what you asked it to do.

2. Re-observe the real state of the files. Do not trust your memory of the plan you had before the interruption. Look at what's actually on disk right now: which files changed, what the diff contains, whether tests pass, whether the change matches what you expected. Your pre-interruption mental model of "it was probably about halfway through step 3" is not evidence — it's a guess, and often a wrong one, because agents don't always work through a task in the linear order you imagine. Measure, don't remember.

3. Redispatch only the measured residual. Once you know, from actual evidence, what's done and what isn't, redispatch a new brief that covers only the remaining work — with the prior agent's report included as context, so the new agent isn't starting from zero on a task that's already 80% finished. Redoing the entire task "to be safe" is the single most expensive mistake available here: it throws away real, verified work, and it risks a second write clobbering a first write that was actually correct.

4.3 Why "redo the whole thing" feels safer than it is

It's worth sitting with why the instinct to restart from scratch is so strong, because understanding the pull helps you resist it. Restarting feels safe because it doesn't require you to trust anything about the interrupted state — you don't have to verify what happened, you don't have to reason about partial completion, you just wipe the slate and start clean. That feeling of safety is an illusion built on skipping the one step that actually determines whether restarting is safe: checking whether the first attempt already produced something correct that a restart would overwrite or duplicate.

If the interrupted agent had already, say, correctly migrated forty of sixty files, "redo the whole thing" doesn't just waste the effort spent on those forty — it risks a second pass making a different (and possibly worse) decision about files that were already handled correctly, now with no record of what the first pass's reasoning was. The measured residual approach costs you a few minutes of investigation. The restart approach can cost you hours of untangling two overlapping, inconsistent attempts at the same work.

4.4 A worked example

You dispatch an agent late in the day to migrate your project's test suite from one assertion library to another across twenty test files — a mechanical but non-trivial task, since each file's assertions need individually correct translation, not a blind find-and-replace. Your laptop goes to sleep mid-session. When you reopen it the next morning, the conversation shows the dispatch but no completion report.

Wrong recovery: assume the migration never happened or is unreliable since it was interrupted, and dispatch a fresh agent to "redo the test suite migration from scratch," without checking anything first.

Right recovery:

1. Check whether the original agent's task is still listed as running, or whether it has a terminal status you can retrieve. Suppose you find it completed — it has a report, you just never saw it land.
2. Read that report. Suppose it says: seventeen of twenty files migrated and passing, two files skipped because their assertion style didn't map cleanly to a mechanical translation and needed a judgment call the agent flagged rather than guessed at, and one file the agent didn't reach before finishing its work.
3. Re-observe the actual repository state: run the test suite, confirm the seventeen claimed-migrated files really do pass under the new library (per Chapter 5, you verify the one load-bearing claim yourself rather than trusting the report blindly), and confirm the three remaining files are genuinely untouched.
4. Dispatch a new, narrow brief: migrate exactly the three remaining files, with the flagged judgment call on the two ambiguous ones stated explicitly as a decision for the new agent to make (or for you to make and hand it), and the prior agent's report attached as context so the new agent isn't rediscovering what the first one already learned about this codebase's assertion patterns.

This costs you the few minutes it takes to check status and read a report. It saves you from re-migrating seventeen files that were already correct, and from a second agent potentially making a different, silently inconsistent choice on those same seventeen files if it had been told to redo everything.

4.5 When the agent itself, not just your connection to it, is the uncertain part

Everything above assumes the agent kept working fine and only your visibility into it broke. Sometimes it's the other way around: your session is intact, but you genuinely don't know whether the agent's underlying process crashed, got rate-limited mid-task, or is still quietly making progress on something slow. The recovery procedure is the same, but step one takes a slightly different shape: instead of hunting for a lost thread, you're checking whether a process you can still see is actually alive and advancing, versus silently dead.

A practical version of this: if a long-running command an agent kicked off (a large test

suite, a slow build, a big file operation) hasn't produced new output in a while, that's not automatically a hang — some legitimate work is genuinely quiet for stretches. Look for advancing evidence before assuming failure: a growing log file, a changing timestamp on an output file, an open network connection still doing something. Advancing evidence with no new console output is latency, not a hang, and killing it prematurely just forces you to start that slow step over from zero. No advancing evidence at all, over a period long enough that the step should have produced something by now, is a genuine suspected hang — and that's when the one-nudge rule from Chapter 3 applies: check it, kill it if it's truly wedged, and move on with a measured understanding of what did and didn't complete, rather than a guess.

4.6 The forward test

A useful way to check whether you've internalized this chapter: when an interrupted commission comes back to you, can you name, from actual evidence rather than assumption, exactly which parts are confirmed done, which are confirmed not-done, and which are genuinely unknown? If your answer to any of those three categories is "I'm assuming," go measure it before you dispatch anything else. The whole discipline in this chapter is substituting five minutes of measurement for however many hours a wrong assumption costs once two independent, uncoordinated attempts at the same work start colliding.

The next chapter turns from the agent's side of the exchange to yours: once a report does arrive — on time, complete, no interruption — what should you actually do with it before you believe it?

Chapter 5

Chapter 5: Receiving a Report

5.1 The report is all that comes back

Once an agent finishes, the report it writes is, for all practical purposes, the entire result of the work as far as anyone downstream is concerned. You may be able to go re-read its full working transcript if something seems off, but nobody actually does that by default — the report is what gets read, what gets acted on, and what gets forwarded to whoever asked you for the update. This makes the discipline of receiving a report at least as important as the discipline of writing the brief that produced it. A brilliant brief followed by an uncritically accepted report gets you exactly as far as a sloppy brief — the failure just shows up later, after you've already acted on something false.

5.2 Send back anything missing verbatim evidence

If you asked for exact counts and exact output in Chapter 1's brief, and the report comes back with "the tests are passing now" instead of the actual command and its actual output, send it back. This is not bureaucratic box-checking — it's the only real check you have on the gap between "I did work that plausibly relates to success" and "I confirmed the specific outcome you asked for." A rounded-up summary is not evidence; it's a claim. Don't accept the claim in place of the number you asked for.

5.3 Absence of counter-evidence is not proof of absence

Watch for a specific, easy-to-miss slip in how reports phrase negative findings. "I didn't find any references to the old API" and "the old API has no remaining references" are different claims, and a report should tell you which one it's making. The first is a statement about the search that was performed — its scope, its method, its limits. The second is a stronger claim about the actual state of the codebase, and it's only as strong as the search behind it. An agent that grepped three directories and found nothing has evidence for the

first claim, not the second — and a report that blurs the two is quietly asking you to accept more certainty than the underlying work actually earned.

5.4 "I delegated this" is not a result

Agents sometimes delegate part of their own task to a further subagent — this is common and often reasonable — and then report back "I delegated the deep dive on X to a specialist." That sentence is a status update about *process*, not a finding. It tells you what happened next, not what was learned. Wait for the actual result before you act on anything; treating the delegation notice itself as a completed answer is how a half-finished chain of work gets reported up as done at every level.

5.5 Verify the one load-bearing claim yourself

You cannot re-verify everything an agent tells you — that would defeat the purpose of dispatching work in the first place. But before you tell anyone else (a teammate, a stakeholder, your own future self reading notes later) that a task is done, pick the single claim the rest of your confidence rests on and check it yourself. Run the test suite. Load the page. Open the file and read the actual line. This isn't distrust of the agent specifically — it's the same discipline you'd apply to any report from any source, human or otherwise, before forwarding it as fact. The cost is small: one command, one click, one file open. The alternative cost — forwarding an unverified "it works" that turns out not to — is much larger, and it lands on your credibility, not the agent's.

5.6 Calibrate ceremony to actual consumers

A related but distinct discipline: don't demand more process than the situation warrants. If you're a solo developer or a very small team, a one-line change with a single caller doesn't need a deprecation period, a compatibility shim, or a phased rollout plan — that ceremony exists to protect real external consumers who can't all update at once, and manufacturing it for a change nobody but you depends on just slows you down without protecting anyone. The skill here is proportion: verify the claim that matters, and don't manufacture process weight the actual stakes don't require.

5.7 "Blocked" needs to say what a competent outsider would try

When a report comes back saying a task is blocked, the report is only useful if it tells you *why*, in a way you can evaluate. A report that says "couldn't do X because our tool for X doesn't support it" is telling you about a limitation in your own specific setup — it is not telling you that X itself is impossible. A useful blocked report names what a competent person, unconstrained by your particular tooling, would try next, and then states specifically why your actual constraints rule that out. "Our internal wrapper for sending email doesn't support attachments" is a fact about the wrapper. It says nothing about whether email with attachments is achievable — it just means the report should say that, rather than implying the whole approach is a dead end.

5.8 Reconciling multiple reports before acting on any one

When you've dispatched more than one agent — following Chapter 2's rules for a legitimate fan-out — their reports will land separately, sometimes close together, sometimes over hours. Resist the urge to act on the first one that arrives. Wait for all of them (or decide explicitly that you're proceeding without one that's overdue, and say so), then reconcile the full set into one coherent picture before you act.

Reconciliation means: deduplicate findings that different agents happened to both surface, and — this is the part people skip — where two agents genuinely disagree about something, record the disagreement rather than silently picking whichever report you happened to read first or trust more on gut feel. A real disagreement between two independent investigations is itself a finding worth keeping, not a mess to be smoothed over by picking a winner without evidence.

5.9 A worked example

You dispatch two agents to investigate a production incident: one to check whether a recent deploy correlates with the start of elevated error rates, one to check whether a third-party API the service depends on had a documented outage in the same window.

Agent A reports: "Deploy timestamps don't line up cleanly with the error spike — the spike started about eleven minutes before the deploy started." Agent B reports: "The third-party API's own status page shows a partial outage starting nine minutes before your error spike, lasting eighteen minutes."

A rushed read might treat Agent A's report as ruling out the deploy and move straight to blaming the third-party outage. A careful reconciliation instead holds both findings side by side: they don't contradict each other, they corroborate the same conclusion from two independent angles — the timing evidence and the external status page both point away from the deploy and toward the outage. Before accepting that conclusion, verify the one load-bearing claim yourself: pull up the third-party status page directly and confirm the outage window with your own eyes, rather than trusting Agent B's paraphrase of it. That one check is cheap. Building a postmortem on an unverified paraphrase, and later discovering the status page said something subtly different, is not.

5.10 A second worked example: the confidently wrong "no issues found"

Consider a different shape of report than the incident-reconciliation example above: you dispatch an agent to check whether a recent dependency upgrade introduced any breaking API changes into your codebase, and it comes back with "no issues found — the upgrade is safe."

Before accepting this, apply the distinction from earlier in this chapter. "No issues found" could mean the agent ran your full test suite, it passed completely, and it additionally diffed the dependency's changelog against your actual usage of its API and found no removed or changed signatures you rely on — a strong claim, well-supported. It could also mean the agent ran a quick grep for a few obviously renamed functions, found none of those specific names, and stopped there — a much weaker claim wearing the same confident sentence.

The report you should actually accept states which one it did, and shows the verbatim evidence for it: which commands were run, what the test suite's exact pass count was, and — if a changelog diff was performed — what specifically was compared against what. If the report just says "no issues found" with no evidence trail, that's not a finding, per this chapter's opening rule — send it back and ask for the verbatim evidence and the explicit scope of what was actually checked before you rely on it for something as consequential as shipping a dependency upgrade to production.

5.11 What this buys you

None of this is about distrusting the agents you dispatch. It's the same discipline you'd want applied to any report you receive from any source before you act on it or pass it along

— the discipline just matters more here because dispatching several agents in parallel means several reports landing at once, each one a candidate to be accepted uncritically under time pressure. The next two chapters turn from single interactions to the layer above them: how you carry what you've learned forward between sessions, and what actually happened, in one real incident, when this whole discipline was tested by two agents colliding on one shared repository.

Chapter 6

Chapter 6: Receipts as Inter-Session Memory

6.1 The problem receipts solve

If you run agent sessions across multiple days, you'll hit this problem within a week: you sit down, open a new session, and you cannot remember — and neither can the agent — what got established yesterday. Did you already check whether the rate limiter handles concurrent requests correctly? Did you decide to use the old date format or the new one? Was that bug actually fixed, or did you just confirm it existed and run out of time?

A chat transcript technically contains the answer, buried somewhere in however many thousand words of back-and-forth. But nobody re-reads a full transcript to answer a specific question — it's too slow, and the answer you need is mixed in with a lot of exploration, dead ends, and conversational filler that isn't the answer. The fix is not "write better notes to your future self in the chat." The fix is a different kind of artifact entirely: a receipt.

6.2 What a receipt is, and what it isn't

A **receipt** is a dated record of an outcome — what was checked, what was found, what changed, what's still open — written so that a later session (yours or a teammate's) can read it instead of re-deriving the same state from scratch. It is not a transcript. A transcript is a log of how a conversation went — every exploratory dead end, every correction, every "wait, let me check something else first." A receipt strips all of that away and keeps only the outcome.

This distinction matters because the two serve different purposes and should not be conflated. If you write a receipt that includes every step of the investigation rather than just its conclusion, you've written a transcript with extra formatting — it's just as slow to extract the answer from as the original conversation was.

A receipt is also distinct from a **living document** — a note, a guide, a piece of reference material that gets revised in place as your understanding improves. A receipt, once written, is a record of a specific moment: what was true, or what was done, on that date. It doesn't get edited later to reflect new understanding; a new receipt gets written instead, dated later, that supersedes or extends it. This is the same reason financial receipts don't get retroactively edited — you don't erase Tuesday's receipt because Wednesday you learned something new; you write Wednesday's receipt and let both stand as an honest record of what was known when.

6.3 The minimal shape of a receipt

You don't need special tooling to start doing this — the discipline matters more than the format. A minimal receipt needs four things:

- **What was checked.** The specific investigation, command, or review that was performed.
- **What was found.** The actual outcome — not a summary that rounds up, per Chapter 5's rules about verbatim evidence.
- **What changed.** If the session resulted in an edit, a fix, a decision — state it plainly.
- **What remains open.** The honest boundary of what this receipt does *not* settle, so the next session doesn't assume more was resolved than actually was.

A receipt this size takes a few minutes to write and can save an hour of re-derivation the next time someone — including you — needs to know what happened.

6.4 A worked example

Suppose you spent an afternoon investigating why a background job occasionally processes the same record twice. Here's the difference between a transcript-shaped note and a real receipt.

Transcript-shaped (don't do this):

> Looked into the duplicate-processing bug today. First checked if it was a database locking issue, spent a while reading the ORM docs, turned out that wasn't it. Then wondered if it was a retry-logic problem, dug into the queue consumer code, found some retry logic but it looked correct at first glance. Eventually noticed the job's idempotency key was being generated from a timestamp that isn't unique enough under load. Didn't fix it yet, need to think about whether

to use a UUID instead or add a database constraint.

Receipt (do this instead):

> **2026-08-14 — duplicate background-job processing investigation** Checked: queue consumer's retry logic and the job's idempotency-key generation, against three days of production logs showing 40 duplicate-processed records. Found: idempotency keys are generated from a millisecond timestamp, which collides under concurrent load — confirmed by reproducing two duplicate keys in a local load test at 50 concurrent jobs. Changed: nothing yet — this is a finding, not a fix. Open: decide between a UUID-based idempotency key or a database uniqueness constraint on the existing key; either fixes the root cause, but a constraint would surface the collision as a visible error instead of silently retrying, which may be the better failure mode to choose deliberately rather than by default.

Notice that the receipt is shorter than the transcript-shaped note, not longer — cutting the exploratory narrative doesn't just make it faster to read, it makes the actual finding easier to find. A teammate (or you, two weeks later) picking this up can decide the open question in thirty seconds of reading, instead of reconstructing the whole afternoon's detour through ORM docs and retry-logic code that turned out not to be the answer.

6.5 When a reconciled finding set is itself a receipt

Chapter 5 covered reconciling multiple agents' reports into one coherent picture before acting on them. That reconciled picture is itself exactly the kind of thing that should become a receipt, written before the session ends — not left to be re-derived by whoever picks up the work next by reading through several separate agent transcripts and re-reconciling them a second time. If you did the work of resolving a disagreement between two investigations, or combining three partial findings into one conclusion, write that conclusion down as a receipt. The alternative — leaving the reconciliation implicit in your own head, or scattered across several separate reports — means the next person (or your own next session) either has to redo the reconciliation or, worse, acts on just one of the un-reconciled partial reports without realizing it was only part of the picture.

6.6 Falsehood is removed, never annotated

One more rule belongs in this chapter because it governs how receipts and living documents interact once something turns out to be wrong. When a document — a receipt, a

guide, a piece of reference material — turns out to contain something false, the fix is to replace the false statement with the true one. Not to append a correction note on top of it. Not to add a bracketed "[UPDATE: this turned out to be wrong]" and leave the original text sitting there for a reader to trip over. Not to build a running log of "verified on this date" stamps next to every claim.

This matters for a concrete reason, not just tidiness: a document that accumulates correction-notes and provenance footnotes becomes genuinely harder to read accurately over time. A reader has to figure out which of several conflicting statements is the current truth, when the document could have simply stated the current truth once. A receipt is different from a living document here — a receipt, being a record of a specific date, doesn't get corrected in place; a new receipt supersedes it. But a living document — the guide, the reference note, the architecture write-up — gets fixed in place: wrong text is deleted and replaced with true text, cleanly, with nothing left behind marking where the wrong version used to be.

6.7 Receipts for decisions, not just findings

Everything above frames a receipt around an investigation — checked, found, changed, open. The same shape works just as well for a decision you made that wasn't strictly a bug hunt: which library you chose and why, which of two architectural approaches you picked, why a price or a deadline landed where it did. These are exactly the things a later session (or a teammate) will otherwise ask about again, from scratch, because the reasoning lived only in your head at the moment you decided.

> **2026-08-20 — choice of idempotency-key strategy** Checked: two options for fixing the duplicate-processing bug — a UUID-based key versus a database uniqueness constraint on the existing timestamp-based key. Found: a uniqueness constraint surfaces the collision as a visible database error on the rare occasions it still happens (useful — it's a signal something upstream is retrying too aggressively), while a UUID key would silently prevent the symptom without ever revealing that the aggressive retries are happening at all. Changed: added the uniqueness constraint, not a UUID key. Deliberately chose the option that surfaces the retry behavior as an error rather than hiding it. Open: the aggressive-retry behavior itself is unaddressed — the constraint fixes the duplicate-processing symptom but the underlying retry logic that causes rapid-fire duplicate attempts still needs its own look.

Notice this receipt does something a bug-fix receipt doesn't need to: it records *why* the

less obviously "complete" fix was chosen on purpose. Six months later, if someone (including you) looks at this code and wonders why a database constraint was preferred over a cleaner-looking UUID approach, the receipt answers it in ten seconds instead of costing someone an afternoon reverse-engineering the reasoning from the commit alone.

6.8 Building the habit

The habit this chapter asks you to build is small and mechanical: at the end of any piece of work worth remembering, spend two minutes writing four sentences — checked, found, changed, open — dated, saved somewhere you (or a teammate) will actually look next time. It will feel like unnecessary overhead the first several times you do it, because in the moment, you remember everything. The payoff shows up two weeks later, when you don't.

The next chapter puts every discipline from this book together in one real, dated incident — not a composite example, an actual collision between two concurrent agent sessions sharing one repository, walked from the moment it went wrong to the moment it was resolved.

Chapter 7

Chapter 7: Case Study — From Collision to Coordination

7.1 A note on this chapter

Every other example in this book is a composite, built to be representative rather than a transcript of one specific event. This chapter is different: it walks a real, dated incident, from a single operator's own engineering log, involving two of their own concurrent agent sessions sharing one repository on the same day. Service names, directory structure, and other identifying detail have been generalized into plain descriptions — a "card-game service," "the router configuration" — because the specifics don't matter to the lesson and aren't this book's to publish. The shape of what happened, the timing, the collision, and the resolution are real.

7.2 The setup

One developer was migrating a fleet of roughly sixteen small web services — mostly single-page game and utility apps — off an aging hosting setup and onto a modern serverless edge platform, one service at a time. This is exactly the kind of task that benefits from running more than one AI coding agent at once: each service's migration is largely independent, the codebase for each is small, and the developer had two concurrent agent sessions open, each working through its own subset of services against the same shared git repository — checked out once, on one machine, with nothing committed yet in either session's working changes.

Neither session had any built-in visibility into what the other was doing. They were two separate conversations, in two separate windows, each with its own agent, both pointed at the same uncommitted working tree.

7.3 The collision

One session (call it Session A) had already migrated five services successfully: each one now pointed, in the shared routing configuration, at its new home on the edge platform instead of the old dead server. The old routing entries for those five services — no longer needed once the migration completed — were staged for removal as part of finishing that work cleanly.

The second session (Session B), working on a different, unrelated batch of services in parallel, encountered those staged removals while reviewing the routing configuration for its own purposes. Its dispatched agent, seeing routing entries that had changed since the last commit with no explanation visible from Session B's own vantage point, judged them to be **drift** — an unexplained, unexpected difference between what the configuration claimed and what a prior known-good state showed — and "fixed" it. It reverted the staged removals and redeployed the routing configuration to what it understood to be the last-known-correct state.

This is the failure worth sitting with carefully, because the agent's individual reasoning was not obviously wrong from where it stood. Reverting an unexplained, unstaged-looking change back to a known-good baseline is a completely reasonable instinct *when the unexplained change really is drift* — some unintended difference nobody meant to introduce. The agent had no way to know, from its own scope, that the change wasn't drift at all: it was another session's legitimate, in-progress, correct work. The moment that revert deployed, all five of Session A's already-migrated services got silently re-pointed back at their old, now-dead origin servers. Five live services went down, not because either agent made an isolated mistake, but because two correctly-functioning agents, working from disjoint views of the same shared resource, produced a genuine collision.

7.4 Why this matches Chapter 2 and Chapter 5, not just bad luck

It's tempting to read this as simply "an agent made a mistake." It's more useful, and more accurate, to read it against the disciplines already covered in this book. This was, in effect, an *unplanned* fan-out: two agents were operating concurrently against overlapping territory — the shared routing configuration — without either session's brief stating that boundary explicitly, and without the disjoint-scope rule from Chapter 2 being enforced on a shared file that both sessions' work happened to touch. And it's a sharp illustration of a distinction from Chapter 5: an unfamiliar, unexplained change in a shared resource is not

automatically drift to be corrected. It might be another party's legitimate, still-in-progress work — and treating "I don't recognize why this changed" as proof that it's wrong, rather than as a signal to ask before acting, is exactly the kind of unverified assumption Chapter 5 warns against, just applied to infrastructure state instead of a report.

7.5 The resolution: division by message

Once the outage was noticed — five services suddenly serving errors instead of their expected pages — the two sessions did not resolve this by one side unilaterally deciding it was right and re-reverting the other's revert, which would have simply restarted the same collision in the opposite direction. They resolved it by direct communication and an explicit division of custody:

- **Session A kept custody of the shared routing configuration.** Since its work was the one affected, and it had the clearest picture of what the correct end state should look like, it took ownership of re-trimming the routing entries correctly and redeploying to restore the five services.
- **Session B ceded routing custody entirely** for the remainder of the work, keeping only the two services it had legitimate, exclusive ownership of, and froze any further edits to the shared routing configuration going forward.
- **Both sessions exchanged an explicit path manifest** — a plain statement of exactly which files or configuration sections each session owned going forward — so that neither agent, in either session, would again encounter the other's legitimate in-progress state and have to guess what it meant.
- **A set of services neither session had touched yet** were explicitly left alone by mutual agreement, rather than either session assuming it was safe to proceed with them unilaterally now that a collision had already happened once.

7.6 The rule this incident supports

The standing rule that comes out of this incident, stated plainly: when two people or two agent sessions share one uncommitted working tree, **stage only your own paths, never run a broad "add everything" style commit across the whole tree, and treat another session's uncommitted work as evidence of a boundary to respect — not drift to fix.** If something looks wrong and you don't have full context for why, the correct first move is to ask, not to revert. A revert is not a neutral, safe default when you don't know who else might be mid-change on the same resource; it is itself an action with a blast radius, and

in this incident, that blast radius was five live services.

7.7 Applying this to your own concurrent sessions

If you run more than one agent session against a shared repository — the exact situation this whole book is written for — build the path manifest habit *before* you need it, not after your own version of this incident. Before starting two sessions on overlapping work, write down, in plain terms, which files or directories each session owns for the duration of that work. If a session's agent encounters a change to a file outside its own declared territory that it didn't make and doesn't recognize, its default action should be to report the unexplained state and ask, not to "fix" it by reverting. This is a small amount of up-front coordination — a sentence or two exchanged between sessions, or written down in a receipt per Chapter 6 — against the real cost this incident shows: five live services down, and two sessions' worth of work spent untangling what should have been prevented, not repaired.

The next chapter pulls every discipline from Chapters 1 through 7 into one operating checklist, so you have something to actually run against before your own next multi-agent session, rather than having to hold seven chapters in your head at once.

Chapter 8

Chapter 8: Building Your Own Operating Discipline

8.1 Why a checklist, and why now

Seven chapters of individual discipline are only useful if you can actually run them under real conditions — mid-afternoon, three things going at once, a deadline you're behind on. Nobody consults a book chapter by chapter in that moment. What you need instead is a compressed operating checklist you can hold in your head, or pin next to your screen, that captures the decision points from every chapter in the order you'll actually hit them.

This chapter is that checklist, followed by guidance on adopting it gradually rather than all at once, since trying to adopt seven new habits simultaneously is its own way to fail.

8.2 Before you dispatch

- **Is this dispatch actually warranted?** Can the result change your next decision, or do you already know what you'll do regardless of what comes back? (Chapter 1)
- **If you're considering more than one agent: does the split change the answer?** Run the three-part test — nameable lots, genuinely disjoint, independently-needed evidence — before fanning out. "One agent per directory" is not a plan by itself. (Chapter 2)
- **Does the brief state all four required things?** Outcome, exact scope, permitted operation, and what "done" measures. Read your own brief back as if you were a stranger who will never talk to you again. (Chapter 1)
- **If more than one agent will touch a shared resource — code, config, infrastructure — have you drawn the boundary explicitly?** Write the path manifest before you need it, not after a collision teaches you to. (Chapter 7)

8.3 While an agent is running

- **A steering message queues for the next tool call, not instantly.** If something urgent changed underneath the agent, send the message immediately, but don't assume it has taken effect until you see the agent's next action confirm it. (Chapter 3)
- **Never say "done" before it's true on disk** — to the agent, to a teammate, to yourself. (Chapter 3)
- **A stuck agent gets one nudge, not a string of them:** check the process, kill it if hung, try once in the foreground, report the real outcome, stop. (Chapter 3)
- **Stopping a parent doesn't stop its children.** If you needed to stop an agent because its plan was wrong, check for and stop anything it spawned separately. (Chapter 3)

8.4 When something interrupts you

- **Absence of a report is not evidence of failure.** Don't assume, and don't start a replacement agent on the same files before checking. (Chapter 4)
- **Find the original agent and its actual current state first.** Terminal report if it finished, live status if it's still running. (Chapter 4)
- **Re-observe the real file state — don't trust your memory of the plan.** (Chapter 4)
- **Redispatch only the measured residual**, with the prior report as context. Redoing everything "to be safe" is usually the most expensive option on the table, not the safest one. (Chapter 4)

8.5 When a report lands

- **No verbatim evidence, no acceptance.** Send it back for the exact command and exact output if it's missing. (Chapter 5)
- **"I didn't find X" and "X doesn't exist" are different claims** — know which one the report is actually making. (Chapter 5)
- **"I delegated this" is a status update, not a result.** Wait for the real finding. (Chapter 5)
- **Verify the one load-bearing claim yourself** before you forward "it works" to anyone else. (Chapter 5)
- **Calibrate ceremony to actual consumers** — don't manufacture process weight a one-caller change doesn't need. (Chapter 5)
- **A "blocked" report should name what a competent outsider would try**, and why your specific setup rules it out. (Chapter 5)

- **Multiple reports get reconciled before any of them get acted on** — dedupe, and record genuine disagreements rather than silently picking a side. (Chapter 5)

8.6 At the end of a session

- **Write the receipt: checked, found, changed, open.** Four sentences, dated, stored where you'll actually look next time. (Chapter 6)
- **A reconciled finding set is a receipt too** — write it down before the session ends rather than leaving it to be re-derived. (Chapter 6)
- **If something you wrote earlier turned out to be false, fix it in place.** Delete the wrong statement and replace it with the true one — no correction notes, no append-only pile of "verified on" stamps on a living document. (Chapter 6)

8.7 Adopting this gradually

Trying to run all of the above from day one, on every task, is a reliable way to give up on it within a week — the overhead feels large before the habit is automatic, even though it's genuinely small once it is. A more durable path:

Week 1: the brief. Just the four elements from Chapter 1 — outcome, scope, permitted operation, done-measure — on every dispatch, nothing else. This alone eliminates the single most common failure mode (confident, wrong work from an incomplete brief) and is the highest-leverage habit in the book.

Week 2: receiving. Add the discipline of demanding verbatim evidence and verifying the one load-bearing claim yourself before forwarding a result. This is where you'll start catching things the brief alone doesn't prevent.

Week 3: receipts. Start writing the four-sentence receipt at the end of any session whose findings you'd regret losing. You'll feel the payoff the first time a receipt saves you from re-deriving something you'd otherwise have forgotten.

Week 4 and after: concurrency discipline. Once the first three habits are automatic, the fan-out test, steering rules, recovery procedure, and path-manifest habit from Chapters 2, 3, 4, and 7 become the layer that protects you specifically when you're running more than one agent at once — which, if you started this book because you already do that, is the layer you were probably most impatient to reach. It's placed last in this adoption sequence anyway, because the single-agent disciplines underneath it are what make concurrent operation survivable in the first place; skipping straight to running several agents at once

with an undisciplined brief and no receiving discipline just multiplies the first three failure modes instead of preventing the concurrency-specific ones.

8.8 The test that tells you it’s working

You’ll know this discipline has actually taken hold, not just been read about, the first time one of the following happens and you notice yourself doing the right thing by reflex, not by consulting a checklist: you catch yourself about to accept a rounded-up “tests pass” and ask for the number instead; you catch yourself about to dispatch a fourth agent on a task that one agent would have finished just as well; you come back from an interruption and check status before redispatching anything. The checklist above is scaffolding. The goal is for the scaffolding to become unnecessary — for the discipline to just be how you operate.

Chapter 9 collects twelve common situations into quick, scannable diagnoses you can check a live problem against without rereading this whole book. The appendix that follows it collects the five templates referenced across this book — the brief template, the fan-out decision checklist, the recovery runbook, the minimal receipt format, and the coordination protocol — in one place, ready to copy and adapt to your own work.

Chapter 9

Chapter 9: Field Diagnoses — Common Situations, Quickly Placed

The previous eight chapters build the discipline from first principles. This chapter is a different kind of reading: short, situation-shaped entries you can scan when something specific has already gone wrong (or is about to), to place it against the right chapter fast, without re-reading the whole book. Each entry names the symptom, the likely cause, the fix, and where to read the full reasoning.

9.1 "The agent said it fixed the bug, but the bug is still there."

Likely cause: the report was accepted without verbatim evidence — a summary like "fixed the failing test" was taken at face value instead of the exact command output that would have shown whether the fix actually took, or only addressed part of the problem.

Fix: don't accept "fixed" as a terminal state. Require the exact command and its exact output as part of every report, and if a prior report didn't include it, go run the command yourself before trusting the claim further.

Read: Chapter 1 (briefing for verbatim evidence), Chapter 5 (receiving a report without accepting an unverified claim).

9.2 "Two agents wrote to the same file and now it's a mess."

Likely cause: either the fan-out test was skipped (the task was split without confirming the lots were genuinely disjoint), or two separate sessions were run concurrently on overlapping territory without an explicit division of ownership.

Fix: stop both agents. Assess the actual current state of the file — not what either agent claims it did — and manually reconcile it once, by hand if necessary, before dispatching

anything else against that file. Going forward, draw an explicit boundary (a path manifest) before running concurrent work on any shared resource.

Read: Chapter 2 (the fan-out test's disjointness requirement), Chapter 7 (the real incident this exact failure produced), the coordination-protocol template in the appendix.

9.3 "I came back to my laptop and I don't know if the agent finished."

Likely cause: an interrupted session with no report received — this is not a sign that anything went wrong, only that the loop between you and the agent broke.

Fix: do not dispatch a replacement yet. Find the original agent (and anything it may have spawned), get its actual current status or terminal report, and re-observe the real file state before deciding what, if anything, still needs doing.

Read: Chapter 4, in full — this is exactly the situation it's written for.

9.4 "The agent kept saying 'still waiting' for twenty minutes."

Likely cause: either a genuinely hung process the agent is polling without noticing it's dead, or a build/test step that's legitimately slow and the agent's repeated status messages just look like a stall.

Fix: apply the one-nudge rule once — check whether the underlying process is actually alive, kill it if it's hung, try the step yourself in the foreground once, and act on the real answer. Don't send another "are you still there?" hoping for a different result than the first one got.

Read: Chapter 3, "The one-nudge rule for a stuck agent."

9.5 "I stopped the agent, but something is still editing my files."

Likely cause: the stopped agent had spawned a child agent to handle part of its task, and stopping the parent didn't stop the child — it kept running as its own independent task.

Fix: check for any tasks the stopped agent may have spawned and stop them individually; don't assume stopping the top-level task stopped everything underneath it.

Read: Chapter 3, "Stopping an agent, and its children."

9.6 "A report says a task is blocked, and I don't know if that's really true."

Likely cause: the report is describing a limitation in your specific tooling or setup and phrasing it as if the underlying goal itself were impossible.

Fix: ask the report to name what a competent person, unconstrained by your particular setup, would try next — and then state specifically why your actual constraints rule that out. "Our wrapper for X doesn't support this" is a fact about the wrapper, not proof that X can't be done.

Read: Chapter 5, "'Blocked' needs to say what a competent outsider would try."

9.7 "I have six agent reports and I don't know what to believe."

Likely cause: several agents' findings landed close together and got acted on individually, in arrival order, instead of being reconciled into one coherent picture first.

Fix: hold all the reports together before acting on any single one. Deduplicate overlapping findings, and where two reports genuinely disagree, write down the disagreement explicitly rather than picking whichever one you happen to trust more without evidence for the choice.

Read: Chapter 5, "Reconciling multiple reports before acting on any one."

9.8 "I can't remember what I already figured out about this bug three days ago."

Likely cause: the prior session's findings live only in a chat transcript, which nobody rereads in full to answer a specific question, rather than in a short, dated receipt written at the time.

Fix: going forward, write the four-line receipt — checked, found, changed, open — at the end of any session whose findings you'd regret losing. For the immediate problem, there's no shortcut: you'll need to either re-derive it or search the transcript this once, and let that cost be the reminder to write the receipt next time.

Read: Chapter 6, in full.

9.9 "A document in my notes contradicts itself — one section says X, another says the opposite."

Likely cause: something in the document turned out to be wrong at some point, and instead of being replaced, the correction got appended as a new note or bracketed aside next to the original false statement, leaving both versions sitting in the same document.

Fix: delete the false statement and replace it with the true one. Don't leave a "correction" marker next to the original — a reader shouldn't have to figure out which of two conflicting statements is current.

Read: Chapter 6, "Falsehood is removed, never annotated."

9.10 "I'm about to split a big task across five agents — should I?"

Likely cause: not yet determined — this is the moment to check, before it becomes a cause of anything.

Fix: run the three-part test before dispatching anything: can you name each of the five lots individually, is each one genuinely disjoint from the others, and does each one need independently-gathered evidence rather than a different slice of the same search? If any answer is no, collapse it to fewer agents, or one.

Read: Chapter 2, in full, and the fan-out decision checklist in the appendix.

9.11 "The agent asked to confirm something I thought I'd already told it."

Likely cause: either the brief genuinely left that detail out (a gap worth closing next time), or the agent is correctly declining to guess at something ambiguous instead of silently picking an interpretation — which is the behavior you actually want, even though it feels like friction in the moment.

Fix: answer the question, then check whether the brief could have included that answer up front. If the same kind of question keeps recurring across similar tasks, that's a signal to add it as a standing element of your brief template rather than answering it fresh every time.

Read: Chapter 1, "The four things every brief needs."

9.12 "A teammate's agent reverted something my agent had just finished."

Likely cause: exactly the shape of the real incident in Chapter 7 — two concurrent sessions, no explicit division of a shared resource, and one agent treating the other's legitimate work as unexplained drift to be corrected.

Fix: don't let either side re-revert unilaterally. Settle it by direct message: agree who has custody of the contested resource going forward, restore what was lost, and write down an explicit path manifest before either of you touches that resource again.

Read: Chapter 7, in full, and the coordination-protocol template in the appendix.

9.13 Using this chapter

None of these twelve entries is a substitute for the reasoning in the chapter it points to — they're a fast way to place a live situation against the right discipline when you don't have time to reread eight chapters from scratch. If a situation you hit isn't listed here, the underlying test still applies: what does the brief that started this task actually say, what evidence backs the claim you're about to act on, and have you measured the real state of things or are you working from memory and assumption? Every entry above, worked backward, reduces to one of those three questions.

Chapter 10

Chapter 10: Scaling This Discipline to a Team

Everything up to this point has been written for one operator running one or more agents. Most of it holds unchanged the moment a second person joins — the brief still needs the same four elements, a report still needs the same verbatim evidence, a receipt is still four sentences. But a few things genuinely change shape once more than one human is involved, and this closing chapter is about those.

10.1 The brief becomes a shared artifact, not a private habit

When you're the only person dispatching agents, a complete brief lives in your own head as a habit — you might not even write it down as a formal document each time, because you're both the author and the sole reader of your own shorthand. Once a second person starts dispatching agents against the same codebase, that shorthand stops working. Person A knows what "the usual scope boundaries" means; Person B, reading a brief Person A wrote quickly, doesn't have that context and will fill the gap with a guess, exactly the way an agent would.

The fix is not more process for its own sake — it's making the brief template from the appendix an actual habit both people use, consistently enough that a brief either of you writes is legible to the other without a conversation first. If your team is small, this can be as simple as a shared convention: every dispatch brief lives in a shared note or a PR description, in the same shape as the appendix brief template, before the agent is dispatched. The goal isn't bureaucracy. It's making sure the same completeness that protects you from a subagent's guesswork also protects your teammate from yours.

10.2 Path manifests stop being optional

Chapter 7's case study happened between two sessions run by the same person on the same day. With two different people running concurrent agents, the odds of an unplanned collision go up, not down — you don't have the advantage of being the same mind that might, by luck, avoid stepping on your own toes. The path-manifest habit from the appendix stops being a nice-to-have for careful operators and becomes close to a requirement: before two people start concurrent agent-assisted work on a shared repository, a two-line message — "I've got the payments service, you've got notifications" — costs nothing and prevents exactly the incident Chapter 7 describes.

Make the default assumption explicit on a team: an unfamiliar, unexplained change in a shared file is someone else's legitimate in-progress work until proven otherwise, never assumed drift to be silently corrected. State this as a team norm, not just an individual habit, because the failure mode it prevents doesn't respect who's "supposed" to know better.

10.3 Receipts become the team's shared memory, not just yours

A receipt written for your own future reference is useful. A receipt written where a teammate can find it is a different order of useful — it's how a team avoids two people independently re-deriving the same finding, or one person's afternoon of investigation dying with them when they're out sick the day someone else needs the answer. If your team doesn't already have a shared, searchable place for dated receipts — even something as simple as a shared folder of dated markdown files, or a channel where receipts get posted in a consistent format — that's worth setting up before your team's agent usage scales much further. The four-field format from Chapter 6 doesn't change; only its audience does.

10.4 Reconciling reports across people, not just across agents

Chapter 5 covered reconciling multiple agents' reports before acting on any one of them. On a team, this extends one level further: when two people have each independently dispatched agents that touched related territory, their conclusions need the same reconciliation treatment before either gets treated as settled. This is a genuinely easy step to skip, because each person feels confident in their own agent's report — the discipline is remembering that "my agent's report" and "the actual state of the system" are not the same thing, exactly as in Chapter 5, and that two people's agents can disagree with each other the

same way two agents dispatched by one person can.

10.5 What doesn't change

It's worth being explicit about what stays exactly the same, because the temptation on a team is to reach for heavier process than the situation actually needs. You do not need a formal agent-orchestration platform, a dedicated approval workflow, or a role called "agent operations lead" to run this discipline on a small team. The four-element brief, the three-part fan-out test, the one-nudge rule, the three-step recovery procedure, and the four-line receipt are all still exactly as lightweight as they were for one person — they just now get written down and shared instead of held in one person's head. Calibrate the ceremony to your team's actual size, per Chapter 5's rule about proportion: a three-person team sharing a Slack channel for receipts is enough process. A three-person team doesn't need a change-management board for agent dispatches.

10.6 A team using more than one kind of agent

A specific version of the coordination problem shows up once a team is big enough that different people have settled on different agentic coding tools — one person runs Claude Code, another runs Codex, a third uses something else entirely. It's worth being explicit that nothing in this book is tool-specific. The brief's four elements, the fan-out test, the steering behavior (a message queuing for the next tool call rather than interrupting mid-action), the recovery procedure, the receiving discipline, and the receipt format all describe how to operate *any* agent that works by reading a brief, taking multiple autonomous steps, and reporting back — the underlying shape every current agentic coding tool shares, whatever its interface looks like.

This matters for a team decision that otherwise causes unnecessary friction: don't standardize on one tool merely to make this discipline easier to teach. The discipline transfers. What doesn't transfer automatically is the coordination layer — a path manifest exchanged between a Claude Code session and a Codex session needs to be just as explicit as one exchanged between two sessions of the same tool, because neither tool has any built-in awareness of what the other is doing to the same shared repository. If anything, a team running more than one agentic tool concurrently should treat the path-manifest habit as more important, not less, since the two tools' sessions have zero shared context with each other beyond what a human explicitly writes down and passes between them.

10.7 The single habit worth building first, on a team

If you're introducing this discipline to a team that's never had it, the brief is still the highest-leverage place to start — the same as it was for one operator in Chapter 8's adoption sequence. A team that gets one thing right — every dispatch brief states outcome, scope, permitted operation, and what "done" measures, in a shared, legible place — has already prevented the majority of the failure modes this book exists to name. Everything after that is refinement on top of a foundation that was already doing the most important job.

Chapter 11

Glossary

Plain-language definitions for every term this book uses in a specific sense. None of these are specialized jargon — they're ordinary words this book pins down precisely so a chapter doesn't have to re-explain itself every time it uses one.

Agent. A large-language-model-driven tool that can read files, run commands, write code, and take multiple steps toward a goal without a human approving each individual action. This book assumes you're already using one (Claude Code, Codex, or something comparable) and focuses entirely on how to run more than one well.

Subagent. An agent dispatched by another agent (or by you, through your primary agent) to handle a bounded piece of work. A subagent starts with no memory of the conversation that dispatched it — see Chapter 1.

Dispatch. The act of handing a bounded task to an agent or subagent to work on, typically without your continuous supervision of every step.

Brief. The complete instruction set you hand an agent when you dispatch it. See Chapter 1 for what a complete one requires.

Scope. The exact boundary of what an agent is permitted to read, investigate, or change — ideally named as specific files or directories, never a vague topic.

Permitted operation. The explicit statement of how far an agent may act: read-only investigation, a proposed patch for review, or an authorized edit. See Chapter 1.

Fan-out. Dispatching more than one agent in parallel against pieces of a larger task. Legitimate when the pieces are genuinely disjoint and independently evidenced (Chapter 2); an anti-pattern when the split is cosmetic — the same task divided for headcount rather than because it changes the answer.

Steering. Sending a message to an already-running agent to correct or redirect it. Takes effect at the agent's next tool call, not instantly. See Chapter 3.

Interrupted commission. A dispatched task whose report never reached you — because a session restarted, a connection dropped, or you simply lost track of it — whose absence is

not evidence that the underlying work failed. See Chapter 4.

Residual work. The measured, still-unfinished portion of an interrupted task, as distinct from the whole original task — the thing you redispatch after recovery, per Chapter 4, instead of starting over.

Report. The output an agent returns when it finishes (or stops) working on a dispatched task. Treated, per Chapter 5, as a claim to be checked against evidence, not an automatic truth.

Verbatim evidence. An exact command and its exact, literal output — a test count, a line count, an exit code — as opposed to a paraphrased summary that may round a partial result up to a full one.

Receipt. A dated, short record of an outcome — what was checked, found, changed, and left open — written so a later session or person can read it instead of re-deriving the same ground truth. See Chapter 6. Distinct from a transcript (a full log of how a conversation went) and from a living document (reference material revised in place over time).

Living document. A note, guide, or piece of reference material that gets corrected in place as understanding improves — the wrong statement is deleted and replaced with the true one, never annotated with a correction note left sitting beside it.

Drift. An unexplained difference between what a system or configuration claims and what it's actually doing — as distinct from another person's or session's legitimate in-progress work, which is not drift merely because you don't yet know why it changed. See Chapter 7.

Path manifest. An explicit, exchanged statement of which files or directories each person or session owns during a period of concurrent work on a shared repository — how you divide custody without a central lock. See the appendix template.

Concurrent session. Two or more separate agent-driven working sessions operating against the same shared repository or resource at the same time, whether run by the same person or different people.

Done. Whatever fixed, checkable standard you named in the brief — never a vague sense that things seem finished. If "done" wasn't defined in the brief, it isn't defined at all, and any claim of it is a guess.

Chapter 12

Appendix: Five Templates

Five reusable artifacts, one per core discipline. Copy these, strip the bracketed placeholders, and adapt the wording to your own workflow — none of this requires any particular tool, only the habit of filling in every field before you dispatch, steer, recover, receive, or coordinate.

12.1 1. The brief template

```
OUTCOME: [What you need to know or have changed, stated as a result,
not an action.]

SCOPE: [Exact paths, files, or directories. State explicitly what is
OUT of scope, especially if another agent or person has territory
nearby.]

PERMITTED OPERATION: [Choose one — Read-only investigation / Proposed
patch for review / Authorized edit.]

DONE MEANS: [A fixed, checkable standard: an exact command and the
expected kind of output, or an exact acceptance check. Not "make sure
it's good."]

REQUIRED IN THE REPORT: exact commands run, exact output (counts, exit
codes), and an explicit "found X" vs. "did not find evidence of X"
distinction for any negative claim.

RELEVANT CONTEXT THE AGENT CANNOT DISCOVER ON ITS OWN: [Anything you
know that isn't in the code or obvious from the repository — a known
gap, a decision already made, a constraint from outside the codebase.]
```

12.2 2. The fan-out decision checklist

Before splitting a task across more than one agent, answer all three. If any answer is no, do it as one agent instead.

```
[ ] Can I name each lot individually, right now, in one sentence each?

[ ] Is each lot genuinely disjoint — no two agents will read or write
    the same file?

[ ] Does each lot need independently-gathered evidence, not just a
    different slice of the same search?

One sentence: "Splitting this into N agents changes the answer
because ___." If you can't finish this sentence honestly, don't split
it.
```

12.3 3. The recovery runbook

For when a dispatched agent's report never arrived.

```
STEP 1 — LOCATE: Find the original agent (and any children it spawned).
Do not dispatch a replacement yet.
  - Still running? -> observe/steer it; do not start a second writer on
    the same files.
  - Finished? -> read its actual terminal report before touching
    anything it may have written.
  - Failed or vanished? -> note its last known command, output, and
    exactly which paths it had touched.

STEP 2 — RE-OBSERVE: Check the real state of the files/repository right
now. Do not rely on your memory of the pre-interruption plan.
  - What is confirmed done (verified, not assumed)?
  - What is confirmed not-done?
  - What is genuinely unknown, and needs a direct check before you
    proceed?

STEP 3 — REDISPATCH THE RESIDUAL ONLY: Brief a new agent (or resume the
```

```
original) with exactly the unfinished portion, plus the prior agent's
report as context. Do not redo confirmed-complete work "to be safe."
```

12.4 4. The minimal receipt format

```
[DATE] — [one-line description of the work]

CHECKED: [what was investigated, reviewed, or run]

FOUND: [the actual outcome — exact counts/output where relevant, not a
rounded-up summary]

CHANGED: [what was actually modified, fixed, or decided — or "nothing
yet, this is a finding" if applicable]

OPEN: [what this receipt does NOT settle — the honest boundary of what
remains undecided or unverified]
```

12.5 5. The coordination protocol (path manifests)

For two or more people, or two or more concurrent agent sessions, sharing one repository.

```
Before starting concurrent work on a shared repository:

SESSION/PERSON A owns: [exact paths or files — nothing implied, nothing
"probably fine to touch"]

SESSION/PERSON B owns: [exact paths or files, disjoint from A's list]

SHARED / NEITHER OWNS YET: [explicitly name anything both might touch —
these paths are frozen until one side claims them or the work finishes]

RULE IN EFFECT: stage and commit only your own declared paths; never a
broad "add everything" commit across the shared tree. If you encounter
a change outside your own declared territory that you don't recognize
and can't explain, the default action is to ASK, not to revert or
"fix" it. Treat it as someone else's legitimate in-progress work until
```

```
you have direct confirmation otherwise.

ON COLLISION: settle by direct message, not by unilaterally re-reverting
the other side's revert. Re-divide custody explicitly (who owns the
contested resource going forward), restore what was lost, and update
this manifest before resuming.
```

That's the whole discipline. None of it requires a framework, a new tool, or a platform migration — it requires writing four extra sentences before you dispatch, four more before you accept a report, and four more when a session ends. The cost is measured in minutes. The failure modes it prevents — redone work, silently overwritten files, a "done" that wasn't true, five services quietly repointed at dead servers — are measured in hours, at minimum, and sometimes in the credibility of whatever "done" claim you forward next.